UNIVERZITA KARLOVA

Přírodovědecká fakulta

Katedra aplikované geoinformatiky a kartografie

Studijní program:

Geoinformatika, kartografie a dálkový průzkum Země

Jakub Fuchsig, Šimon Kohout

Geometrické vyhledávání bodu

Ústí nad Labem, Zákupy 2025

1 Zadání

Vstup: Souvislá polygonová mapa n polygonů $\{P_1, ..., P_n\}$, analyzovaný bod q.

Výstup: P_i, $q \in P_i$.

Nad polygonovou mapou implementujete Ray Crossing Algorithm pro geometrické vyhledání incidujícího polygonu obsahujícího zadaný bod q.

Nalezený polygon graficky zvýrazněte vhodným způsobem (např. vyplněním, šrafováním, blikáním). Grafické rozhraní vytvořte s využitím frameworku QT.

Pro generování nekonvexních polygonů můžete navrhnout vlastní algoritmus či použít existující geografická data (např. mapa evropských států).

Polygony budou načítány z textového souboru ve Vámi zvoleném formátu. Pro datovou reprezentaci jednotlivých polygonů použijte špagetový model.

2 Zpracované bonusové úlohy

Pro tuto úlohu byly zpracovány níže vypsané úlohy společně s jejich ohodnocením.

Krok	**Hodnocení**
Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu.	10b
Analýza polohy bodu (uvnitř/vně) metodou Winding Number Algorithm.	+10b
Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu.	+5b
Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu.	+5b
Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů.	+2b
Zvýraznění úsech polygonů pro oba výše uvedené singulární případy.	+3b
Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu).	+10b
*Načtení vstupních dat ze *.shp.*	+10b
Celkem splněno:	**55b**

3 Úvod do problematiky

Nejenom v počítačové kartografii, ale i v jiných oblastech geoinformatiky se občas setkáváme s řešením úlohy o vzájemné poloze bodu a polygonu. Tento

problém, který se označuje *Point Location Problem*, je jedním ze základních problémů v této oblasti.

Existují dvě techniky řešení problému:

- *Převedení problému na vztah bodu a mnohoúhelníku* - problém řešen pomocí opakovaného určení polohy bodu vzheledek k mnohoúhelníku. Toto řešení je pomalejší, ale snadnější na implementaci.
- *Planární dělení roviny* - rovina je rozdělena na množinu pásů ci lichoběžníků a z nich vzniká tzv. *trapezoidální mapa*. Toto řešení je výrazně rychlejší, ale její implementace je obtížná, jelikož využívá speciální datové struktury (např. binární stromy).

Do tohoto problému dále zasahují i vlastnosti hledaných polygonů, resp. jejich geometrie. Jedná se o konvexní a nekonvexní polygony. U konvexních polygonů (všechny jejich vnitřní úhly mají maximální velikost 180°) je problematika jednodušší než u nekonvexních. Příkladem nekonvexních polygonů z praxe mohou být např. hranice správních obvodů či přírodní jevy.

Pro řešení tohoto typu problému byly využity dva základní algoritmy *Ray Crossing* a *Winding Number* (Bayer 2026).

4 Filtrování pomocí Min-Max Boxu

Pro optimalizaci a urychlení výpočtů algortimů nad polygony, které mohou mít velké množství vrcholu byla zvolena metoda filtrace pomocí tzv. *Min-Max Boxu*.

Princip spočívá v nalezení extrémních souřadnic daného polygonu. Prochází se všechny jeho vrcholy a hledají se minimální a maximální hodnoty na obou osách:

$$x_{min} = \min(x_i), \quad x_{max} = \max(x_i)$$
$$y_{min} = \min(y_i), \quad y_{max} = \max(y_i)$$

Tyto čtyři hodnoty vytvoří kolem polygonu pomyslný obdélník. Ještě před samotným spuštěním algoritmů je kontrolováno, zda souřadnice analyzovaného bodu $q(x_q, y_q)$ vůbec leží uvnitř tohoto obdélníku. Testuje se následující podmínka:

$$(x_q < x_{min}) \vee (x_q > x_{max}) \vee (y_q < y_{min}) \vee (y_q > y_{max})$$

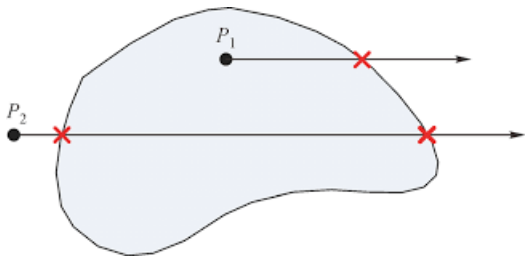
Pokud je tato podmínka splněna, je bod s jistotou mimo polygon a algoritmus může okamžitě výpočet ukončit s návratovou hodnotou -2. Pokud bod projde tímto filtrem (nachází se uvnitř obdélníku), přistupuje program k samotnému zvolenému algoritmu. Tento krok snižuje výpočetní čas při prohledávání rozsáhlých dat.

5 Ray Crossing Algorithm

Principem této metody je polopřímka r (paprsek, tzv. *Ray*), která je vedena naším zvoleným bodem q. Poté se zkoumá, kolikrát (k) tento paprsek protnul hrany zkoumaného polygonu. Pomocí následujícího rozhodovacího pravidla následně rozhodne, zda je bod vně (k je liché) či uvnitř (k je sudé) polygonu.

$$k\%2 = \begin{cases} 1, & q \in P, \\ 0, & q \notin P \end{cases}$$

Tento princip lze pozorovat i na obrázku 1.



Obrázek 1: Princip Ray Crossing (Wang a Benson 2016)

Pro zjednodušení výpočtů se používá posunutí souřadnicového systému tak, aby počátek náležel právě zkoumanému bodu (x_q, y_q). Nové souřadnice jednotlivých vrcholů polygonu se vypočítají následovně:

$$x_i' = x_i - x_q$$
$$y_i' = y_i - y_q$$

Jelikož je paprsek definován jako kladná poloosa x je možné odovodit vhodný průsečík x_m s hranou polygonu pomocí následujícího vztahu:

$$x_m = \frac{x'_{i+1}y'_i - x'_i y'_{i+1}}{y'_{i+1} - y'_i}$$

Pokud je výpočet > 0, tak hrana protíná náš paprsek a započítává se průsečík. Pokud je hodnota $= 0$, bod leží přímo na hraně polygonu (Bayer 2026).

Co se týče **singularit**, tedy bod leží přesně **na hraně** polygonu či **ve vrcholu**. Pokud bod leží na hraně, pak $x_m = 0$ a v našem algoritmu hned vracíme hodnotu -1, tedy "bod leží na hraně". Pokud se jedná o situaci, kdy je bod totožný s vrcholem, tedy $x'_i = 0$ a $y'_i = 0$, algoritmus okamžitě tuto situaci kontroluje a opět vrací hodnotu -1.

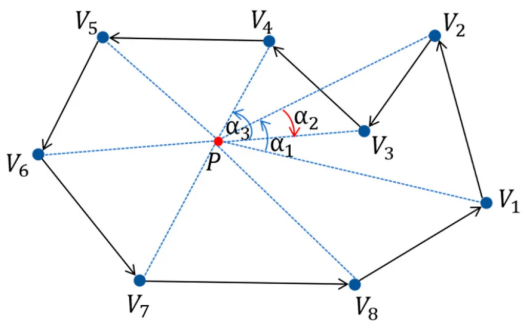
Další možnosti vzniku singularity je, že nás paprsek prochází vrcholem polygonu. V tomto případě by mohl být průsečík započítán dvakrát. I to je kontrolováno tak, aby vrchol vždy náležel pouze k jedné ze dvou sousedních hran a průsečí se započítal pouze jednou, tedy $(y'_{i+1} > 0) \wedge (y'_i \leq 0)$ nebo $(y'_{i+1} < 0) \wedge (y'_i \geq 0)$.

6 Winding Number Algorithm

Tato metoda oproti *Ray Crossing* nepracuje s průsečíky, ale počítá sumu všech rotací Ω. Za tyto rotace lze označit úhly mezi jednotlivými průsečíky bodu q s vrcholy polygonu (obrázek 2). Na tomto principu pak algoritmus rozhoduje, zda bod leží uvnitř či vně zkoumaného polygonu:

$$|\Omega| \approx 2\pi \rightarrow q \in P$$
$$|\Omega| < 2\pi \rightarrow q \notin P$$

Pokud je absolutní hodnota sumy vnitřních úhlů je přibližně 2π, bod se nachází uvniř polygonu. Pokud je hodnota menší, bod by měl ležet mimo polygon či na hraně polygonů (Bayer 2026).



Obrázek 2: Princip Winding Number (Liu et al. 2023)

Ke každému vrcholu polygonu $p_i = (x_i, y_i)$ a $p_{i+1} = (x_{i+1}, y_{i+1})$ jsou z bodu q počítány vektory

$$\vec{v_1} = (x_i - x_q, y_i - y_q)$$
$$\vec{v_2} = (x_{i+1} - x_q, y_{i+1} - y_q)$$

Velikost úhlu α mezi těmito vektory se následně vypočítá pomocí jejich skalárního součinu a euklidovských vzdáleností s využitím funkce arkus kosinus:

$$\alpha = \arccos\left(\frac{\vec{v_1} \cdot \vec{v_2}}{|\vec{v_1}||\vec{v_2}|}\right)$$

Ke správné orientaci sčítání úhlů je potřeba určit, na které straně hrany leží bod q. Lze využít tzv. *cross product*.

$$D = (x_{i+1} - x_i)(y_q - y_i) - (y_{i+1} - y_i)(x_q - x_i)$$

Pokud je $D > 0$, bod je na levé straně hrany a úhly se budou sčítat, pokud je $D < 0$, situace je opačná a úhly se budou odečítat. Z tohoto důvodu je v úvodu napsána $|\Omega|$.

7 Singulární případy

V rámci implementace obou algoritmů bylo nezbytné korektně ošetřit **singulární** případy. Konkrétně se jedná o situace, kdy analyzovaný bod q leží

přímo ve vrcholu polygonu, nebo incidentně leží kdekoli jinde na některé z jeho hran.

7.1 Bod je totožný s vrcholem polygonu

Tato singularita nastává, pokud analyzovaný bod $q = (x_q, y_q)$ splývá s některým vrcholem polygonu $P_i = (x_i, y_i)$. Geometricky to znamená, že euklidovská vzdálenost mezi těmito dvěma body je rovna nule, neboli velikost směrového vektoru $\vec{v}_1 = P_i - q$ je nulová. Výpočet velikosti vektoru je definován vztahem:

$$\|\vec{v}_1\| = \sqrt{(x_i - x_q)^2 + (y_i - y_q)^2} = 0$$

Pokud je tato podmínka splněna (případně i pro vektor $\vec{v}_2$ směrující k následujícímu vrcholu), algoritmus detekuje singularitu a okamžitě vrací hodnotu -1.

7.2 Bod leží na hraně polygonu

Předpokládejme hranu polygonu definovanou vrcholy P_i a P_{i+1}. Analyzovaný bod q tvoří s těmito vrcholy dva vektory:

$$\vec{v}_1 = P_i - q$$
$$\vec{v}_2 = P_{i+1} - q$$

Pokud bod q leží přesně na úsečce mezi vrcholy, pak jsou vektory $\vec{v}_1$ a $\vec{v}_2$ kolineární (leží v jedné přímce) a směřují do opačných směrů.Z kolinearity vektorů ve 2D prostoru vyplývá, že jejich determinant (neboli z-tová složka vektorového součinu) musí být roven nule:

$$\det(\vec{v}_1, \vec{v}_2) = v_{1x}v_{2y} - v_{1y}v_{2x} = 0$$

Opačná orientace vektorů geometricky znamená, že úhel α, který tyto vektory svírají, je roven přímému úhlu, tedy π radiánů (180°). Velikost tohoto úhlu je v algoritmu odvozena ze skalárního součinu obou vektorů:

$$\cos\alpha = \frac{\vec{v}_1 \cdot \vec{v}_2}{\|\vec{v}_1\|\|\vec{v}_2\|} \quad \Longrightarrow \quad \alpha = \arccos\left(\frac{v_{1x}v_{2x} + v_{1y}v_{2y}}{\|\vec{v}_1\|\|\vec{v}_2\|}\right) = \pi$$

Eliminace zaokrouhlovacích chyb. Vzhledem k tomu, že výpočet probíhá v oboru reálných čísel a využívá výpočetně náročné goniometrické funkce,

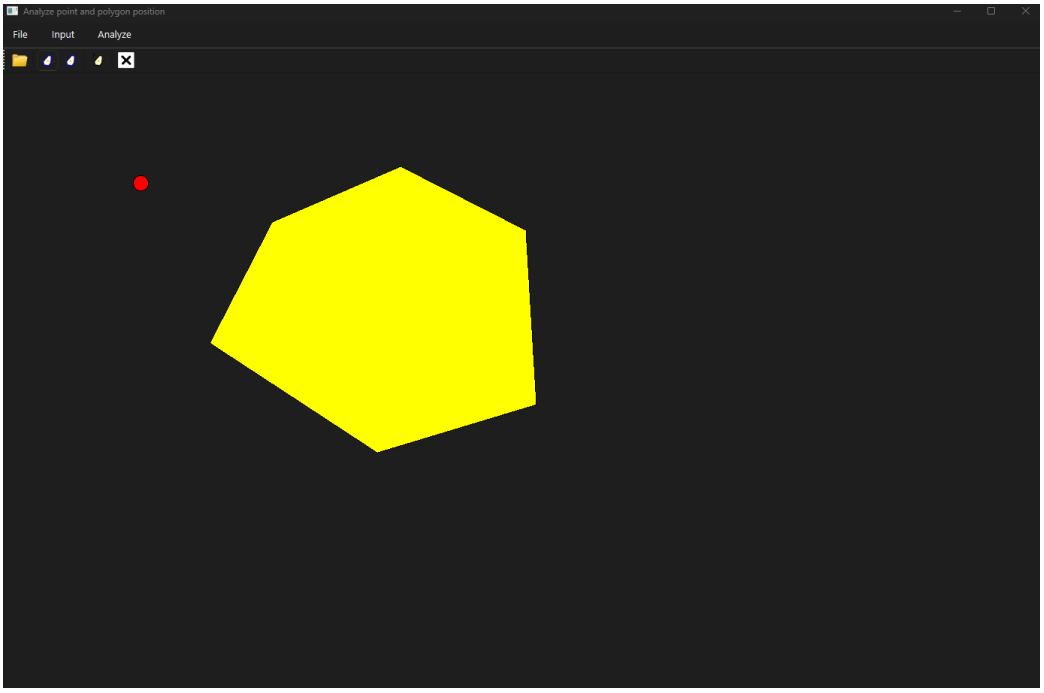
dochází při strojovém zpracování k drobným zaokrouhlovacím chybám. Rovnost s nulou nebo s hodnotou π tedy nelze testovat absolutně. Pro robustní detekci hrany je zavedena toleranční mez $\epsilon = 1.0 \cdot 10^{-7}$. Singulární případ hrany je algoritmem potvrzen pouze tehdy, platí-li:

$$|\det(\vec{v}_1, \vec{v}_2)| < \epsilon \quad \wedge \quad |\alpha - \pi| < \epsilon$$

8 Popis aplikace

Pro analýzu polohy bodu a polygonu byla vytvořena kostra aplikace pomocí programu *QT Designer*. Tato kostra byla dále upravena v programovacím jazyce Python. V této části byla v různých případech použita AI z důvodu úpravy syntaxe či navrhnutí workflow (např. pro nahrání souboru .shp) a případného vysvětlení či objasnění problému.

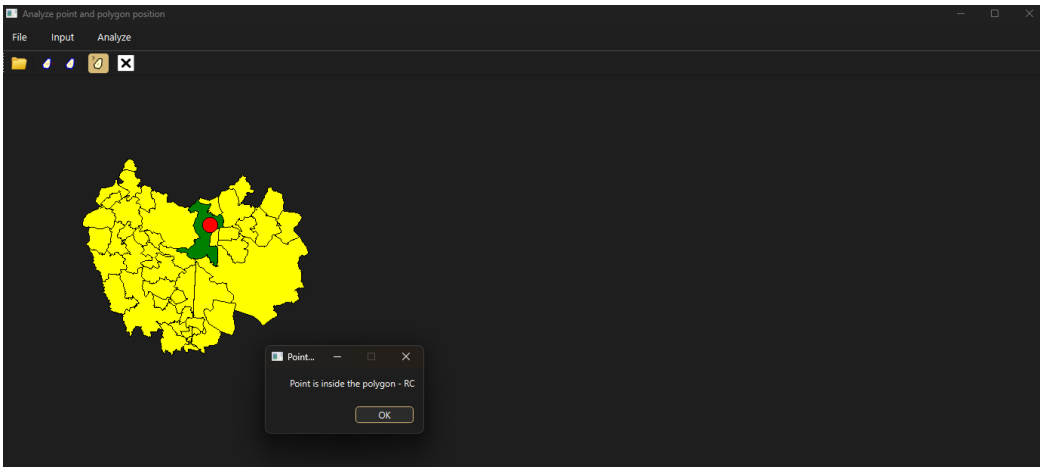
Uživatel má možnost výběru dvou výše zmíněných algoritmů k určení pozice bodu a polygonu. Jednotlivé akce jsou spustitelné pomocí ikon v liště aplikace (obrázek 3). Uživatel si může pomocí kursoru vytvořit vlastní polygon či při přepnutí módu (pomocí ikony *Point/Polygon*) udávat pozici bodu. Pokud má uživatel vlastní soubor .shp, lze ho nahrát pomocí tlačítka *File -> Open*.



Obrázek 3: Okno aplikace s ručně vytvořeným polygonem

Vstupními daty mohou být tedy i data ve formátu .shp. Uživatel si může vybrat vlastní soubor z lokálního uložiště a nahrát ho do aplikace. Algoritmus vypočítá maximální šířku a výšku celého území a vypočítá vhodné měřítko pro zasazení polygonů na plátko aplikace. Aby se polygony vykreslily správně, bylo nutné souřadnice jednotlivých vrcholů přepočítat pomocí měřítka a také posunout pomocí *offsetu*. Tento krok byl důležitý, jelikož počátek plátna náleží hornímu levému rohu okna. Každý přepočítaný polygon je převeden na objekt *QPolygonF* a uložen do seznamu, ze kterého byly objekty vykresleny na plátno.

Výstupem aplikace je správně určený polygon, ve kterém se nachází hledaný bod q (zobrazen červenou tečkou). Takto určený polygon je následně obarven zelenou barvou s příslušnou hláškou podle zvoleného algoritmu (obrázek 4).



Obrázek 4: Výsledek detekce bodu

Pomocí **číselné interpretace** byly zajištěny různé výsledky z algoritmů, které sloužily jako doplňkové informace o řešení polohy bodu a polygonu. Každý algoritmus po dokončení analýzy vrací jednu ze čtyř celočíselných hodnot, díky které obarvuje polygony a vypisuje uživateli příslušné hlášení do vyskakovacího okna:

- **Hodnota 1 (Uvnitř):** Testovaný bod se nachází bezpečně uvnitř polygonu. Polygon je překreslen tmavě zelenou barvou.
- **Hodnota 0 (Vně):** Testovaný bod se nachází mimo polygon. Polygon zůstává ve výchozí žluté barvě.
- **Hodnota -1 (Singularita):** Testovaný bod leží přesně na hranici polygonu (na hraně nebo přímo ve vrcholu). Aplikace vytvoří specifickou

hlášku a polygony obarví azurovou barvou.

- **Hodnota -2 (Filtrováno):** Testovaný bod leží zcela mimo opsaný obdélník (Min-Max Box). Algoritmus vůbec neprováděl výpočet a rovnou bod vyloučil. Polygon zůstává žlutý.

9 Popis tříd a jejich metod

9.1 Třída *MainForm*

Tato třída je hlavním třídou aplikace. Stará se o okno aplikace a propojení tlačítek s funkcemi.

- **setupUi** - Inicializuje a rozmisťuje grafické prvky v okně aplikace a propojuje je s příslušnými funkcemi.
- **changeStatusClick** - Reaguje na kliknutí tlačítka a mění mod mezi kreslením polygonu a změnou pozice testovacího bodu.
- **clearClick** - Maže veskeré vykreslené polygony a body.
- **analyzePointAndPositionClick** - Spouští analýzu polohy bodu pro všechny vykreslené polygony pomocí algoritmu *Ray Crossing* a otevře vyskakovací okno s výsledkem.
- **analyzeWindingNumberClick** - Spouští analýzu polohy bodu pro všechny vykreslené polygony pomocí algoritmu *Winding Number* a otevře vyskakovací okno s výsledkem.
- **retranslateUi** - Nastavuje textové popisky v uživatelském rozhraní.
- **openFile** - Otevírá okno s cestou na disku k volbě .shp souboru.
- **loadShapefile** - Načte vybraná .shp soubor, přepočítá souřadnice vrcholů a převede do seznamu polygonů.

9.2 Třída *Algorithms*

V této třídě se vyskytují výpočty a algoritmy.

- **getPointPolygonPositionRC** - Určuje polohu testovaného bodu vůči zadanému polygonu pomocí algoritmu *Ray Crossing* a vrací výsledek ve formě kódu (1, 0, -1 nebo -2).

- **getPointPolygonPositionWN** - Určuje polohu testovaného bodu vůči zadanému polygonu pomocí algoritmu *Winding Number* a vrací výsledek ve stejné formě.

9.3 Třída *Draw*

Tato třída se stará o vykreslování samotného plátna, interakci s kursorem a vykreslení dat.

- **mousePressEvent** - Zaznamenání kliknutí myší do plátna a podle modu vykreslí vrchol polygonu nebo změní pozici bodu.
- **PaintEvent** - Zajišťuje vykreslení polygonů a bodu na plátno. Na základě výsledku z algoritmu přebarví příslušný polygon zeleně, modře nebo žlutě.
- **changeStatus** - Přepíná mezi režimem přidávání bodu k polygonu či změnu pozice bodu.
- **clearData** - Vymaže plátno a nastaví výsledky do původního stavu.
- **getQ** - Vrací aktuální sořadnice testovaného bodu pro výpočty algoritmů.
- **getPols** - Vrací seznam všech existujících polygonů.
- **setPols** - Nahradí aktuální polygony na plátně novými, vybranými ze .shp.
- **setResult** - Uloží příslušný výsledek analýzy a indexu polygonu a požádá o překreslené polygonu.

10 Závěr

Prostřednictvím funkční desktopové aplikace s grafickým uživatelským rozhraním v prostředí PyQt byly úspěšně implementovány algoritmy Ray Crossing a Winding Number pro řešení problematiky prostorového vztahu bodu a polygonu (Point Location Problem). Součástí řešení je i ošetření singulárních případů, kdy analyzovaný bod leží přímo na hraně polygonu nebo v jednom z jeho vrcholů. Aplikace v těchto situacích poskytuje uživateli okamžitou textovou i vizuální zpětnou vazbu pomocí barevného zvýraznění detekovaného polygonu. Pro optimalizaci a zrychlení výpočtů nad rozsáhlejšími datovými sadami, které lze do aplikace importovat ve formátu Shapefile (*.shp), byla

navíc úspěšně aplikována metoda filtrace potenciálních polygonů pomocí tzv. Min-Max Boxu.

Přestože předložená aplikace splňuje hlavní požadavky zadání, lze najít několik oblastí, které by bylo možné dále rozšířit a vylepšit. Aktuální verze programu využívá základní datovou reprezentaci polygonů, která neumožňuje zpracování komplexních geometrií, jako jsou polygony s vnitřními dírami nebo. Možnost importu vstupních polygonů je aktuálně omezena pouze na formát Shapefile, tudíž by bylo přínosné rozšířit podporu i o další moderní geoprostorové formáty (např. GeoJSON) a umožnit načítání více vrstev nad sebou.

Z hlediska uživatelského rozhraní a celkové interakce by vhodným vylepšením byla implementace nástrojů pro posouvání a přibližování (pan & zoom) plátna, na kterém jsou polygony vykresleny, což by výrazně usnadnilo orientaci v hustých mapových podkladech. Uživatelskému komfortu by také přispělo plně automatické spouštění analýzy ihned po změně polohy testovaného bodu, bez nutnosti manuálního vyvolání akce přes horní lištu. Zajímavým rozšířením by mohla být také přímá grafická vizualizace průběhu algoritmů na plátně, například reálné vykreslení testovacího paprsku u metody Ray Crossing nebo vizualizace jednotlivých vektorů a úhlů u metody Winding Number.

Zdroje

Bayer, Tomáš (2026). *Prezentace Agoritmů počítačové kartografie, Point Location Problem.* URL: `https://web.natur.cuni.cz/~bayertom/images/courses/Adk/adk3_new.pdf`.

Liu, Lei et al. (2023). "Efficient Construction of Voxel Models for Ore Bodies Using an Improved Winding Number Algorithm and CUDA Parallel Computing". In: *ISPRS International Journal of Geo-Information* 12, s. 473. DOI: `10.3390/ijgi12120473`.

Wang, Yingjun a David Benson (2016). "Geometrically constrained isogeometric parameterized level-set based topology optimization via trimmed elements". In: *Frontiers of Mechanical Engineering* 11, s. 1–16. DOI: `10.1007/s11465-016-0403-0`.